TECHNICAL DOCUMENTATION

Fiber Liquidity Layer

Infrastructure, Architecture & Testing Guide

An operability console for a single operator's **CKB Fiber Network** node. It makes channel liquidity visible, pre-flights payments with a *"can I pay?"* probe, and self-heals channel balance through idempotent circular rebalancing — with every money-moving action gated behind wallet-based operator identity and recorded in a balanced double-entry ledger.

3 CORE CAPABILITIES	**47** AUTOMATED TESTS (BACKEND 22 · FRONTEND 25)
5 ARCHITECTURAL HARD-RULES ENFORCED	**1** TYPED RPC SEAM TO THE NODE

Live frontend: https://sluice.drreamer.digital · **API:** https://api.sluice.drreamer.digital

Node: FNN 0.9.0-rc7 (testnet) · pubkey `026eaeae…cc86b26`

Stack: NestJS 11 · Prisma 7 · PostgreSQL · Redis/BullMQ · Next.js 16 / React 19 · CCC wallet connector

1 System Overview

The Fiber Liquidity Layer is a **node operations console**, not a personal wallet. It observes and operates a single Fiber Network node that custodies its own on-chain key server-side. A connected browser wallet is used purely to prove *operator identity*; it never signs node operations and never holds the liquidity shown on screen.

1.1 What it does

Capability	Description	Surface
Liquidity visibility	Live per-channel outbound/inbound, aggregate health, peers, reconciliation drift, and auto-derived operator alerts — streamed from the node.	Canvas dashboard, `/channels`, `/liquidity`, `/network`
"Can I pay?" probe	A `dry_run` pre-flight that asks the node whether a payment is routable, its fee, and its bottleneck — *before* any funds move.	`/probe` · `POST /routing/probe`
Self-healing rebalance	Moves liquidity from an over-funded channel to a depleted one via a circular self-payment — idempotent, concurrency-safe, audit-logged.	`/rebalance` · `POST /rebalance`

> **Design stance.** The node is the single source of truth for balances. The database is an *audit record* of what the tool did and a stale-fallback snapshot — never an authority. Every guarantee in this document flows from that stance.

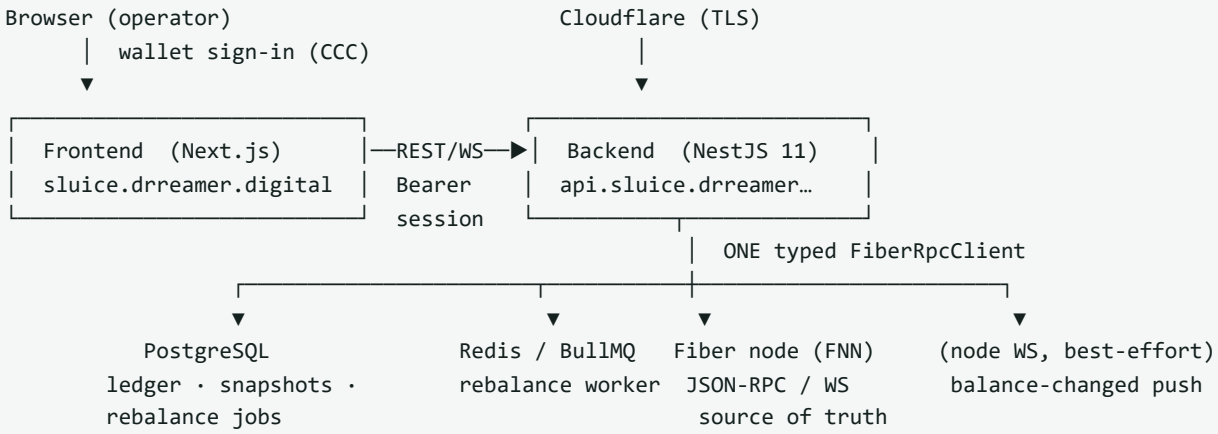
2 Infrastructure Topology

All services run as containers on a single VPS under **Dokploy**, joined on a private Docker network and reached over the internal service DNS. Cloudflare terminates TLS and fronts both public hostnames.

2.1 Components

Component	Role	Reached as
Fiber node (FNN)	Third-party `nervos/fiber` image (0.9.0-rc7). Source of truth for channels, routing, payments. RPC + WebSocket.	`sluice-fiber:8299` (internal)
Backend	NestJS 11 API + orchestrator. The only component that talks to the node. Exposes REST + a realtime WS gateway.	`api.sluice.drreamer.digital`
Frontend	Next.js 16 / React 19 operator console (standalone image). Reads the API; connects a CKB wallet for sign-in.	`sluice.drreamer.digital`
PostgreSQL	Double-entry audit ledger + channel snapshots + rebalance jobs. Prisma 7 with the pg driver adapter.	`postgres-…:5432` (internal)
Redis + BullMQ	Durable queue for the rebalance worker (runs in-process via `RUN_WORKER_INLINE`).	`redis-…:6379` (internal)

2.2 Connection & data-flow map



Reads are live-first from the node; on a node blip the API serves the last snapshot and marks it `stale`. A poller diffs `list_channels` every `POLL_INTERVAL_MS` and pushes `balance-changed` over the frontend WS, so the console updates without a refresh.

2.3 Verified live state (at time of writing)

Signal	Value	Meaning
`node_info`	v0.9.0-rc7 · testnet · 3 peers	Node reachable, gossip healthy
`channels/health`	**source: live** · 2 channels · 9,901 CKB out / 0 in each	Straight from the node, not cached
Total sendable	19,802 CKB	Aggregate outbound capacity
Reconciliation	in-sync · 0 drift	Snapshot matches node

3 Architecture & Design Principles

The backend is a modular monolith of **bounded contexts** (node, channels, routing, rebalance, ledger, reconciliation, realtime, auth), each with the canonical shape `controllers / services / repositories / mappers / dto / types` and a public token surface. Five **hard rules** govern anything that touches money; each is a design decision *and* a testable invariant.

#	Rule	How it is enforced
1	**One typed RPC seam** — all node communication is JSON-RPC/WebSocket through a single `FiberRpcClient` (no gRPC, no Rust, no on-chain code).	Every consumer injects the `FIBER_RPC` token; the adapter is the only place that opens a socket to the node.
2	**Node is the source of truth** — balances are read live from the node, never trusted from the DB.	Health is live-first; the DB snapshot is a labelled `stale` fallback. Reconciliation re-snapshots *from* the node on drift.
3	**Idempotent & concurrency-safe** money ops.	Serializable transaction + a `@unique` idempotency key + P2002/P2034 retry. A duplicate submit returns the same job and never enqueues twice.
4	**Balanced double-entry ledger.**	Every rebalance writes principal-out + fee-out + principal-in; the write is rejected unless `Σ OUTBOUND = Σ INBOUND + fee`.
5	**No `Number()` on amounts** — u128 exceeds `MAX_SAFE_INTEGER`.	All amounts are parsed/stored as `BigInt` / `Decimal(40,0)` decimal strings end-to-end.

> **Why this matters for testing (§6):** ports-and-adapters, a single RPC seam, pure derivation functions, and Zod DTOs mean the money path can be unit-tested by swapping the seam for a mock — no node, no database, no network required.

4 The Money Path — Rebalance Pipeline

A rebalance is a **circular self-payment**: value leaves the over-funded source channel, routes through the network, and returns through the depleted destination channel — increasing the destination's outbound. The request path is thin; the work runs off it on a queue.

```
POST /rebalance ──▶ RebalanceService.request()
     │  serializable txn + unique idempotencyKey (retry P2002/P2034)
     │  new? → enqueue on BullMQ ; duplicate? → return same job, no enqueue
     ▼
BullMQ worker ──▶ RebalanceExecutor.execute()
     ├─ BUILDING   list_channels + node_info + build_router (egress→ingress)
     ├─ (fee gate) send_payment_with_router dry_run — abort if fee > maxFee
     ├─ INFLIGHT   send_payment_with_router (real; 0 retries — never retry money)
     ├─ poll       get_payment  Created → Inflight → Success | Failed
     └─ settle     serializable txn → job SUCCEEDED + ledger.writeRebalancePair()
```

4.1 Idempotency & concurrency

The job is created inside a `Serializable` transaction keyed by a client-supplied idempotency key. Racing inserts collide on the unique constraint (P2002) or serialization (P2034) and are retried; the worker also early-returns if the job is already terminal. Net effect: **the same rebalance can be submitted many times and executes at most once.**

4.2 Double-entry ledger

On success the executor writes three balanced rows inside the settlement transaction:

Direction	Type	Channel	Amount
OUTBOUND	PRINCIPAL	source	amount
OUTBOUND	FEE	source	fee
INBOUND	PRINCIPAL	dest	amount

The repository refuses to write unless `Σ OUTBOUND = Σ INBOUND + fee`. The ledger is an audit record of tool actions — it is never used as a balance authority.

5 Security Model — Operator Identity

Reads are public (the console is viewable by anyone); **mutations are gated behind wallet-based operator identity** using *Sign-In-With-CKB* via the CCC connector (JoyID, MetaMask, and other CKB wallets).

5.1 Sign-In-With-CKB flow

```
1. Frontend  GET /auth/challenge         → single-use nonce embedded in a human-readable message
2. Wallet    signer.signMessage(message) → { signature, identity, signType }   (CCC)
3. Frontend  POST /auth/verify           → { nonce, signature }
4. Backend   Signer.verifyMessage(...)   → verifies server-side (any wallet, incl. JoyID, in Node)
             identity ∈ OPERATOR_KEYS ?   → yes: mint session JWT ; no: 403 echoing the key
5. Frontend  Authorization: Bearer <jwt>  on every mutation
```

5.2 The guard

Request	Decision
GET / HEAD / OPTIONS, `/health`, `/auth/*`	**always public**
Mutation with a valid `Bearer` whose `sub` ∈ `OPERATOR_KEYS`	**allow**
Mutation with `x-dashboard-secret` = `DASHBOARD_SECRET`	allow (break-glass fallback)
Mutation, nothing configured (dev)	allow (dev default only)
Mutation, otherwise	**401**

Signature verification runs entirely server-side via `@ckb-ccc/core` `Signer.verifyMessage`; the JoyID path verifies locally because the public key is embedded in the CCC identity (no browser, no network). The JWT HMAC secret lives only on the backend.

6 How the Design Enables Testing

Testability is a direct consequence of the architecture. Four decisions make the money path verifiable without a running node or database:

1. **Ports & adapters + one RPC seam.** The node is behind a single injectable interface, so services are unit-tested with a mock client — the real node is never needed for logic tests.
2. **Pure derivation functions.** Liquidity math (outbound %, health, rebalance-pair selection, alert rules) and amount conversion (u128 ↔ decimal) are side-effect-free and exhaustively unit-tested.
3. **Dependency injection.** Services receive their collaborators, so a test constructs one with fakes (`new RebalanceService(prismaMock, repoMock, queueMock)`).
4. **Typed DTOs (Zod).** Request shapes validate at the edge, keeping invalid input out of the money path and giving tests a precise contract.

6.1 Automated test suite (47 tests)

Area	What is asserted	Layer
u128 conversions	hex↔decimal round-trips, full 128-bit range without precision loss (never `Number()`).	backend
Double-entry ledger	3 balanced rows with correct directions/types; the `Σ OUT = Σ IN + fee` invariant holds across amounts.	backend
Rebalance idempotency	new key → creates + enqueues once; duplicate → same job, never re-enqueues; P2002 retried; non-retryable rethrown.	backend
Auth service	valid+allowlisted → JWT; valid+not-allowlisted → 403 echoing the key; bad signature → reject; nonce single-use/expiry.	backend
Operator guard	reads & `/auth/*` public; Bearer session allows; expired/foreign token → 401; secret fallback; dev allow-all.	backend
Amount formatting	BigInt-safe CKB formatting/summing beyond `MAX_SAFE_INTEGER` ; truncation; percentages.	frontend
Liquidity derivations	outbound %, channel state, health score, rebalance-pair pick, alert rules from live channel shapes.	frontend

6.2 Continuous integration

A GitHub Actions workflow gates every push/PR to `main` / `staging` : install → backend typecheck + test + build → frontend lint + test + build. Prisma client generation runs on install; no database connection is required for the gate.

7 Testing Guide (Practical)

These checks exercise the live deployment end-to-end. Reads need no auth; mutations need a wallet session.

Test 1 — Node reachability

```
curl -s https://api.sluice.drreamer.digital/node/info
```

Expect: `200`, real `pubkey`, `version 0.9.0-rc7`, non-empty peers. Proves the single RPC seam reaches the node.

Test 2 — Live liquidity

```
curl -s https://api.sluice.drreamer.digital/channels/health
```

Expect: `source: "live"`, `stale: false`, per-channel outbound/inbound as decimal strings. Rule 2 in action.

Test 3 — "Can I pay?" probe AUTH

```
POST /routing/probe  { targetPubkey, amount, maxFee? }
```

Expect signed-out: `401`. **Signed-in:** a verdict `{ payable, fee?, reason?, hops?, bottleneck? }` — a `dry_run` against the real node; no funds move.

Test 4 — Wallet sign-in

1. Open the console → **Connect** → pick a wallet (MetaMask/JoyID) → unlock.
2. Click **Sign in** → approve the plain-text signature (no gas, no transaction).
3. First time: the API returns `403 … Operator key: <signType>:<identity>`.
4. Put that value in `OPERATOR_KEYS`, restart backend, sign in again → header shows **Operator** ✓.

Test 5 — Security gate

Call	Signed out	Signed in
`GET /channels/health`	200	200
`POST /rebalance`	401	202 (accepted)

Confirms reads stay public while money-moving endpoints require operator identity.

Test 6 — Rebalance pipeline (end-to-end, real node)

Signed in, queue a rebalance (source over-funded, dest depleted). **Expect:** accepted `202`, job walks `PENDING → BUILDING → INFLIGHT`, then the executor calls the real node's `build_router` and returns the node's own verdict. This proves the whole pipeline is wired to the live node, authenticated, idempotent, and audit-ready.

Test 7 — A settling rebalance NEEDS LIQUIDITY

To reach **SUCCEEDED** the destination channel needs **inbound liquidity** and a routable loop — impossible while every channel is 100% outbound. Two ways to create it:

- **One outbound payment** through a channel flips it to have inbound; then rebalance the other way (works if the two peers are connected in the graph).
- **A second controlled node** with two channels funded from opposite sides forms a guaranteed 2-hop loop; the rebalance always settles and posts to the ledger.

> **Interpretation.** A `PathFind: no path found` result is a truthful on-chain answer from the node, not a defect — it demonstrates that the pipeline is talking to real routing, not a mock.

8 Known Limitations & Roadmap

Area	Status / next step
Rebalance **settlement**	Pipeline complete and node-integrated; a settled run needs inbound liquidity + a loop topology (2nd node or peer-funded channel). On-chain setup, not code.
Node **funding** flow	Expose `default_funding_lock_script` as the node's CKB receiving address so the browser wallet can top it up from the UI.
Probe as a **read**	Currently a gated POST; could be made public (it is a pure `dry_run`).
Sessions / nonces	In-memory single-instance today; move to Redis for horizontal scale.
Node WS push	`subscribe_store_changes` is method-not-found on rc7, so the backend poller is primary; the WS path is best-effort.

Appendix A — Public endpoints

Method	Path	Auth	Purpose
GET	/health	public	liveness
GET	/node/info · /node/peers	public	identity & peers
GET	/channels/health	public	live liquidity
GET	/reconciliation/status	public	snapshot drift
GET	/auth/challenge	public	sign-in nonce
POST	/auth/verify	public	verify signature → session
POST	/routing/probe	operator	"can I pay?" dry-run
POST	/rebalance	operator	queue circular rebalance
GET	/rebalance/:id	public	job status

Appendix B — Key environment variables

Service	Variable	Purpose
Backend	`FIBER_RPC_URL` / `FIBER_WS_URL`	the single RPC seam to the node
Backend	`DATABASE_URL` · `REDIS_URL`	ledger/snapshots · rebalance queue
Backend	`OPERATOR_KEYS` · `AUTH_JWT_SECRET`	wallet allowlist · session signing
Backend	`CORS_ORIGINS` · `DASHBOARD_SECRET`	pinned origin · break-glass fallback
Frontend	`NEXT_PUBLIC_API_URL` / `_WS_URL`	API location (build-time)
Frontend	`HOSTNAME=0.0.0.0`	standalone server bind (runtime)